Sensors & Systems

Model Uncertainty and Validation

Torben Knudsen | Electronic Systems, 8th Semester

Section 1 Model Uncertainty

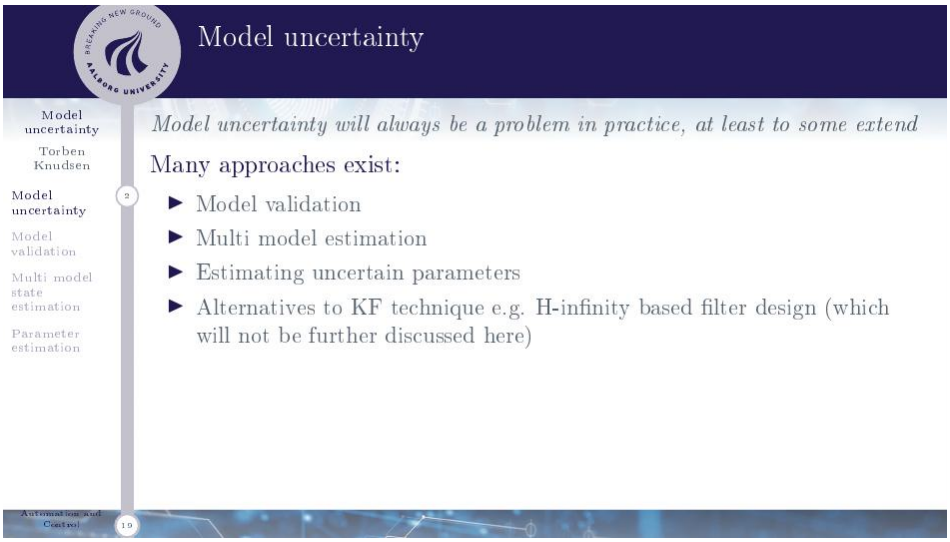

Figure 1: Overview of the approaches to dealing with model uncertainty. The lecture covers the first three; H-infinity filter design is an alternative that is not discussed further here.

Every model used in a Kalman filter, controller, or estimator is an approximation of a real physical system. In practice, some degree of **model uncertainty is unavoidable** — it arises from simplifications made during modelling, parameters that are not precisely known, components that vary between units, or dynamics that change over time.

The key question is not whether uncertainty exists, but **how to handle it**. There are four main strategies:

- **Model validation** — test whether the current model is good enough for its intended purpose, and check statistically whether the residuals (prediction errors) look the way they should if the model were correct. If the model is wrong, this reveals it. *Covered in Section 2.*
- **Multi-model estimation** — when you cannot decide between several candidate models, run all of them in parallel and let the data determine which is most likely correct at each time step. The final state estimate is a probability-weighted combination of all model-specific estimates. *Covered in Section 3.*
- **Estimating uncertain parameters** — if the model structure is correct but some numerical parameters are unknown or slowly drifting, treat those parameters as extra

states and estimate them alongside the physical states using an EKF/UKF, or use system identification methods (maximum likelihood) to find the best-fitting values offline. *Covered in Section 4.*

- **H-infinity filter design** — an alternative to the Kalman filter that does not assume Gaussian noise and instead minimises the worst-case estimation error. Robust to model uncertainty by design, but more conservative. *Not covered further in this lecture.*

Why model uncertainty always exists in practice

When building a state-space model $\dot{x} = f(x, u) + w$, $y = h(x) + v$, every design choice is an approximation:

- *Linearisation:* nonlinear systems are often approximated as linear around an operating point. Away from that point, the model is wrong.
- *Unknown parameters:* spring stiffness, damping coefficients, friction — these are rarely known exactly and vary between physical units of the same design.
- *Changing dynamics:* a vehicle's mass changes as fuel is consumed; a robot arm's inertia changes as it picks up and puts down objects; a battery's internal resistance grows with age.
- *Ignored dynamics:* real systems have resonances, delays, and nonlinearities that are deliberately left out of the model to keep it tractable. Those omitted effects still show up in the data.

None of these can be eliminated entirely. The three strategies in this lecture provide principled ways to detect, quantify, and compensate for these inevitable discrepancies.

Section 1 — Key Takeaways

- Model uncertainty is unavoidable — every model is an approximation. The question is how to detect and handle it.
- Three strategies are covered: **validation** (is the model good enough?), **multi-model estimation** (run competing models in parallel), and **parameter estimation** (estimate what you do not know).
- H-infinity design is a fourth robust alternative but not covered here.

Section 2 Model Validation

Before applying a model to prediction, filtering, or control design, you need to know whether it is fit for purpose. Model validation provides two very different answers to two very different questions.

All three statistical tests use the same sequence

When the Kalman filter runs, it produces at every time step a **residual** (also called the innovation or output prediction error):

$$\tilde{y}_{k|k-1} = y_k - \hat{y}_{k|k-1}$$

This is simply the difference between what you actually measured (y_k) and what the model predicted you would measure ($\hat{y}_{k|k-1}$). After running the filter over your entire dataset you have a full sequence of these residuals: $\tilde{y}_{1|0}, \tilde{y}_{2|1}, \ldots, \tilde{y}_{N|N-1}$.

All three statistical tests in this section analyse that same sequence — just from different angles:

- **Run test** (Section 2.2): are sign changes in the sequence random? A quick check for temporal correlation.
- **ACF / Portmanteau** (Section 2.3): are specific time lags correlated? A more detailed check of the time structure.
- **Normality test** (Section 2.4): is the distribution of the residuals Gaussian? A completely separate question about the shape of the distribution, not about time structure.

You collect the residuals once from one Kalman filter run, and then apply all three tests to that same sequence.

2.1 Two Validation Questions

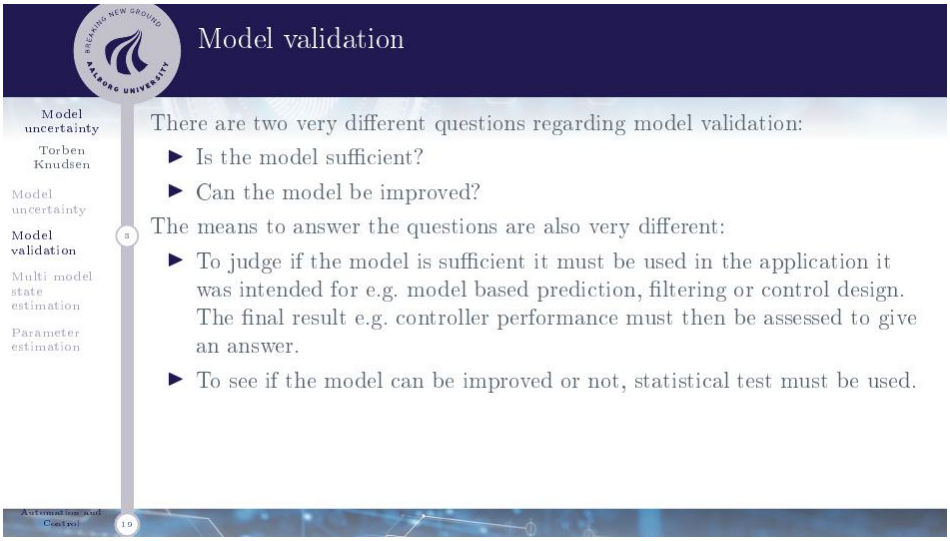

Figure 2: The two model validation questions require completely different tools. Sufficiency is assessed by putting the model to work; improvability is assessed by statistical tests on the residuals.

Question 1: Is the model sufficient? This asks whether the model is *good enough for what you intend to use it for*. The answer depends entirely on the application: a model used for a loose position estimate has a very different accuracy requirement than one used for safety-critical control. The only way to answer this is to *use the model* — run it in the intended application (prediction, filtering, controller design) and evaluate the outcome against your requirements. There is no purely mathematical threshold; it is an engineering judgement.

Question 2: Can the model be improved? This asks whether the data contains systematic structure that the current model is missing — i.e. whether a better model could explain the measurements more accurately. This question *can* be answered statistically, by analysing the **residuals** (prediction errors).

> **Why the two questions need different tools**
>
> Question 1 is about performance in context. Even a model with known errors might be perfectly acceptable if those errors are small relative to what the application needs. You can only know this by trying it.
>
> Question 2 is about the model's internal consistency with the data. It does not depend on the application at all. If a model is correct, its residuals must look like pure random noise — any remaining structure in the residuals is evidence the model could be improved. Statistical tests can detect that structure objectively.
>
> In practice you answer Question 2 first (is there room to improve?), and if the answer is no, you then answer Question 1 (is "good enough" actually good enough for my use case?).

2.2 White Noise Test — Run Test

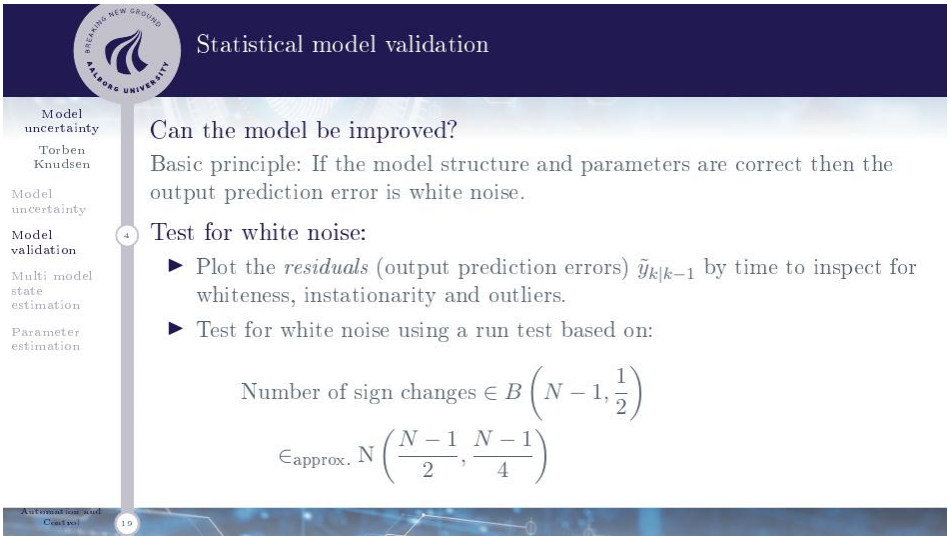

Figure 3: Statistical model validation via a run test on the residuals. The number of sign changes in the sequence $\tilde{y}_{k|k-1}$ is compared against the distribution expected for white noise.

The fundamental principle of statistical validation:

If the model structure and all parameters are correct, then the **output prediction errors** (residuals):

$$\tilde{y}_{k|k-1} = y_k - \hat{y}_{k|k-1}$$

must be **white noise** — a sequence that is zero-mean, has constant variance, and is *uncorrelated* from one time step to the next.

Why must residuals be white noise if the model is correct?

The Kalman filter uses the model to predict the next output $\hat{y}_{k|k-1}$. If the model captures all the dynamics correctly, those predictions account for every pattern in the data — every correlation, every trend, every oscillation. What remains in the residual $\tilde{y}_{k|k-1} = y_k - \hat{y}_{k|k-1}$ is then only the unpredictable part: pure measurement noise. Pure noise has no time structure — it is white.

Conversely, if the residuals *are* correlated (e.g. several consecutive residuals are all positive, or they oscillate in a regular pattern), the model is missing something. There is a pattern in the errors that a better model could predict and remove. The residuals are then *not* white, and the model can be improved.

The run test — testing for white noise via sign changes:

One simple way to test for temporal correlation is to count **sign changes** in the residual sequence. A sign change occurs at step k whenever $\tilde{y}_{k|k-1}$ and $\tilde{y}_{k-1|k-2}$ have opposite signs.

- **Too few sign changes:** residuals stay positive (or negative) for long stretches — they are slowly varying, meaning they are positively correlated. This often means the model is missing a low-frequency trend or a slow dynamic.

- **Too many sign changes:** residuals alternate sign almost every step — they are negatively correlated (anti-correlated). This can indicate the measurement noise variance R is underestimated.
- **The right number:** sign changes are random and unpredictable — consistent with white noise.

Under the null hypothesis (residuals are white noise), the number of sign changes S in a sequence of N residuals follows a **Binomial distribution**:

$$S \in B\big(N-1,\ \tfrac{1}{2}\big)$$

because each adjacent pair is equally likely to have the same or different sign. For large N this is well approximated by a Normal distribution:

$$S \ \approx\ \mathcal{N}\left(\frac{N-1}{2},\ \frac{N-1}{4}\right)$$

Symbol	Name	Meaning
$\tilde{y}_{k\|k-1}$	Residual (prediction error)	Difference between measured output y_k and the model's one-step-ahead prediction $\hat{y}_{k\|k-1}$. Should be white noise if the model is correct.
S	Number of sign changes	Count of how many times adjacent residuals have opposite signs across the full sequence of N residuals.
N	Sequence length	Total number of residual samples.
$B(N{-}1, \frac{1}{2})$	Binomial distribution	Exact distribution of S under white noise. $N{-}1$ trials, each succeeding with probability $\frac{1}{2}$.
$\mathcal{N}\big(\frac{N-1}{2}, \frac{N-1}{4}\big)$	Normal approximation	Mean $= (N{-}1)/2$ (half the adjacent pairs change sign); variance $= (N{-}1)/4$ (since $\mathrm{Var} = np(1-p) = (N{-}1)\cdot\frac{1}{2}\cdot\frac{1}{2}$).

Practical decision: compute S from the data, then check whether it falls inside the 95% interval of the Normal approximation. If S is outside — too high or too low — the white noise hypothesis is rejected and the model can likely be improved.

> **Why you need a long sequence — not just a few samples In short:** run the model over a representative, long operating period and collect residuals from the full run. A short burst of data only tells you about iteration-to-iteration variance, not whether the model systematically misses any dynamics.

2.3 Autocorrelation Test — Portmanteau

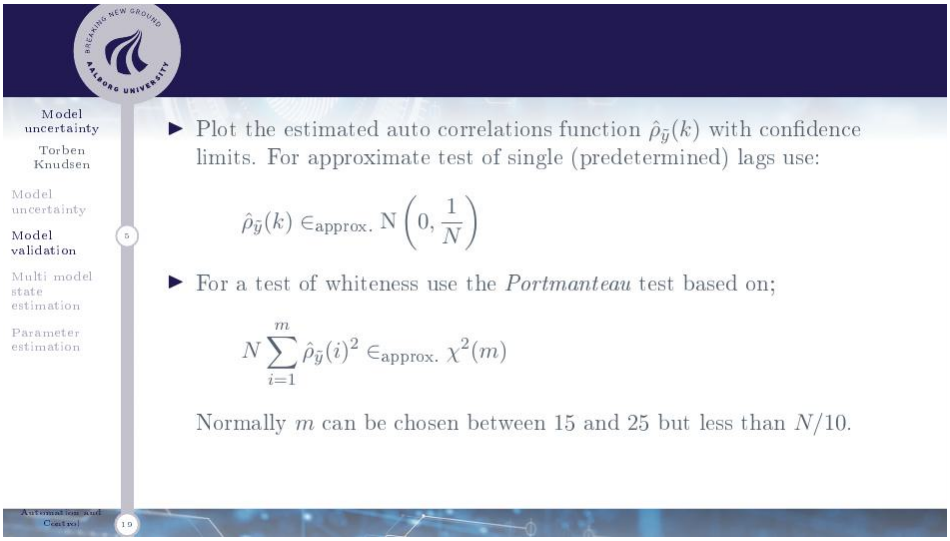

Figure 4: Testing whiteness via the estimated autocorrelation function (ACF). Individual lags can be tested against $\mathcal{N}(0, 1/N)$; the Portmanteau test checks all lags $1 \dots m$ simultaneously.

A more powerful approach is to estimate the **autocorrelation function** (ACF) of the residuals and test whether it is consistent with white noise at multiple lags simultaneously.

The estimated autocorrelation at lag k is:

$$\hat{\rho}_{\tilde{y}}(k) = \frac{\sum_{i=1}^{N-k} \tilde{y}_i \, \tilde{y}_{i+k}}{\sum_{i=1}^{N} \tilde{y}_i^2}$$

For a white noise sequence of length N, each individual lag estimate is approximately:

$$\hat{\rho}_{\tilde{y}}(k) \;\approx\; \mathcal{N}\left(0,\; \frac{1}{N}\right)$$

The standard deviation is therefore $\sigma = 1/\sqrt{N}$. For a $\mathcal{N}(0, \sigma^2)$ distribution, 95% of the probability mass lies within $\pm 1.96\,\sigma \approx \pm 2\sigma$. Substituting:

$$\pm 2\sigma = \pm \frac{2}{\sqrt{N}}$$

So the **95% confidence bounds** for a single lag are $\pm 2/\sqrt{N}$ — these are the horizontal dashed lines on every ACF plot. If a lag estimate falls *outside* these bounds, there is less than a 5% chance of seeing a value that extreme by chance if the residuals truly are white noise. We therefore reject the white noise hypothesis at 95% confidence for that lag, meaning the residuals are significantly correlated at that time separation and the model can likely be improved.

The Portmanteau test — testing all lags at once:

Testing each lag separately inflates the false-alarm rate (if you test 20 lags at 95% confidence, you expect one false positive even for white noise). The **Portmanteau test** (also known as the Box-Pierce test) combines all lags 1 through m into a single statistic:

$$\boxed{N \sum_{i=1}^{m} \hat{\rho}_{\tilde{y}}(i)^2 \;\approx\; \chi^2(m)}$$

If this statistic exceeds the $\chi^2(m)$ critical value (e.g. the 95th percentile), the white noise hypothesis is rejected across the tested lags.

Symbol	Name	Meaning
$\hat{\rho}_{\tilde{y}}(k)$	ACF at lag k	Estimated correlation between residuals k steps apart. Zero for white noise; nonzero indicates structure at that time lag.
N	Sequence length	Number of residual samples.
m	Number of lags tested	How many lags are included in the Portmanteau statistic. Typically chosen between 15 and 25, but must be less than $N/10$ to keep the chi-squared approximation valid.
$\chi^2(m)$	Chi-squared distribution	The distribution of the Portmanteau statistic under white noise, with m degrees of freedom — one per tested lag.

Why $m < N/10$?

The chi-squared approximation for the Portmanteau statistic becomes unreliable when m is large relative to N. At high lags, the ACF estimate $\hat{\rho}_{\tilde{y}}(k)$ is based on only $N - k$ pairs of data points, so it becomes noisy and unreliable. The rule $m < N/10$ ensures that even the highest tested lag still uses at least $0.9N$ pairs — enough for the Normal approximation of each $\hat{\rho}$ to hold, which in turn makes the χ^2 approximation for the combined statistic valid.

In practice, $m = 20$ is a common default for moderate-length sequences.

2.4 Normality Test

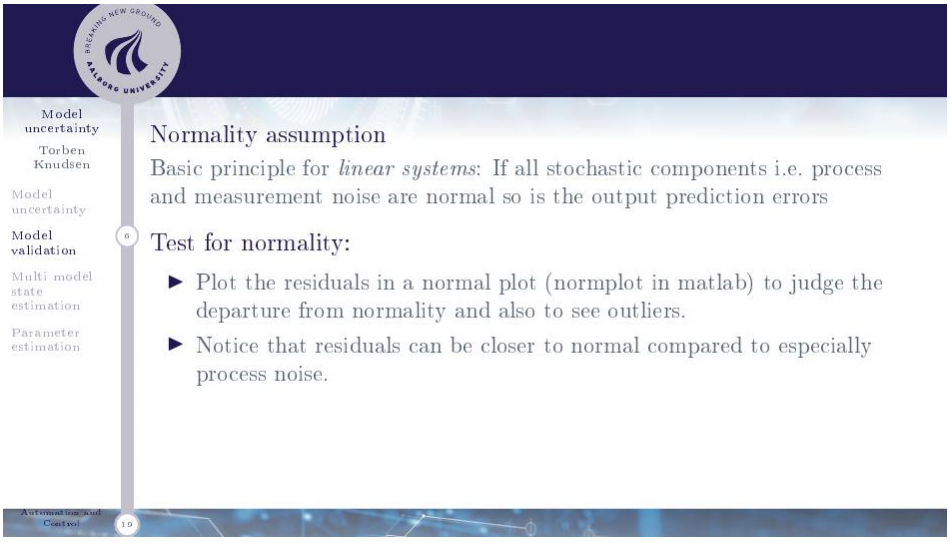

Figure 5: Normality assumption: for linear systems with Gaussian noise, residuals should also be Gaussian. A normal probability plot (normplot) reveals departures from normality and identifies outliers.

The Kalman filter is optimal only when both process noise w_k and measurement noise v_k are Gaussian. For **linear systems**, if the noise inputs are Gaussian then the output prediction errors $\tilde{y}_{k|k-1}$ are also Gaussian — since a linear transformation of a Gaussian is still Gaussian.

Testing normality: plot the sorted residuals against the quantiles of a standard normal distribution — a **normal probability plot** (`normplot` in MATLAB).

- If the residuals follow a straight line on the normal plot, they are approximately Gaussian — the normality assumption holds.
- Curves at the tails indicate heavier or lighter tails than Gaussian (e.g. t-distributed noise).
- Individual points that deviate sharply at the extremes are **outliers** — single large errors not consistent with Gaussian noise. These may indicate sensor faults, sudden disturbances, or incorrect data.

Why residuals are often more Gaussian than the raw noise

The residual $\tilde{y}_{k|k-1}$ is not the raw measurement noise v_k in isolation — it is the output of the full Kalman filter, which involves a weighted combination of many past measurements and the model predictions.

By the **central limit theorem**, linear combinations of many independent random variables tend toward a Gaussian distribution, regardless of the distribution of the individual variables. This means the residuals can look quite Gaussian even if the underlying process noise w_k is not — the averaging inside the filter smooths out non-Gaussian tails.

Practically: a normplot on the residuals can look good (approximately straight) even when the system noise is non-Gaussian, so this test provides a necessary but not sufficient check on the Gaussian assumption.

Section 2 — Key Takeaways

- There are two distinct validation questions that are answered completely differently:
  - **Sufficient?** — just use the model for its intended purpose (run the controller, run the filter, run the predictor) and check whether the result meets your requirements. No statistics needed — it is purely an engineering judgement call.
  - **Improvable?** — this is where the statistical tests (run test, ACF, Portmanteau, normality) come in. You never need to "try the model in use" for this; you only need the residuals from running the Kalman filter on your data.

  A model can pass all statistical tests and still be insufficient for a demanding application — or fail them and still be good enough for a loose one.
- If the model is correct, residuals $\tilde{y}_{k|k-1} = y_k - \hat{y}_{k|k-1}$ must be **white noise** — uncorrelated, zero mean. Correlated residuals mean the model is missing structure.
- **Run test:** count sign changes S. Under white noise $S \sim B(N-1, \frac{1}{2}) \approx \mathcal{N}\left(\frac{N-1}{2}, \frac{N-1}{4}\right)$. Too few or too many sign changes rejects the white noise hypothesis.
- **ACF / Portmanteau:** estimate $\hat{\rho}_{\tilde{y}}(k)$ for lags $1 \dots m$. Individual lags $\sim \mathcal{N}(0, 1/N)$; combined: $N \sum_{i=1}^{m} \hat{\rho}_{\tilde{y}}(i)^2 \sim \chi^2(m)$. Choose m between 15–25 and $m < N/10$.
- **Normality:** use a normal probability plot. Residuals are often more Gaussian than raw noise due to the averaging inside the Kalman filter.

Section 3 Multi-Model State Estimation

Sometimes you do not know which model is the correct one. Rather than guessing and committing to one model, the multi-model approach runs *all candidate models simultaneously* and lets the incoming data determine which is most likely correct.

3.1 Motivation — When Multiple Models Arise

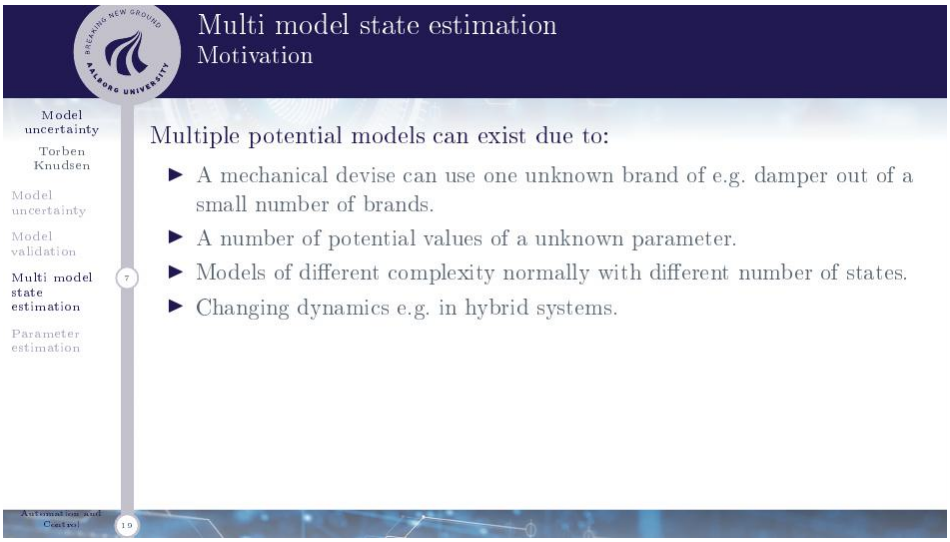

Figure 6: Four common reasons why multiple candidate models coexist in a real system.

Multiple plausible models arise in practice for four main reasons:

- **Unknown component brand:** a mechanical device contains a damper from one of a small set of known suppliers, but you do not know which one was installed. Each supplier's damper has a different damping coefficient. You have J candidate models, one per supplier.
- **Unknown parameter value:** a parameter in the model is unknown but belongs to a small discrete set of possible values. For example, the payload mass of a robot arm could be 0 kg, 1 kg, or 2 kg depending on what it is carrying. Each possible value gives a different model.
- **Different model complexity:** a simple first-order model and a more complex second-order model might both be plausible descriptions of the same system. Rather than choosing upfront, you run both and let the data decide.
- **Changing dynamics (hybrid systems):** the correct model switches between modes as operating conditions change. A vehicle driving on dry road uses one friction model; on ice it switches to another. The transition times are unknown.

3.2 Static Case — Setup and Definitions

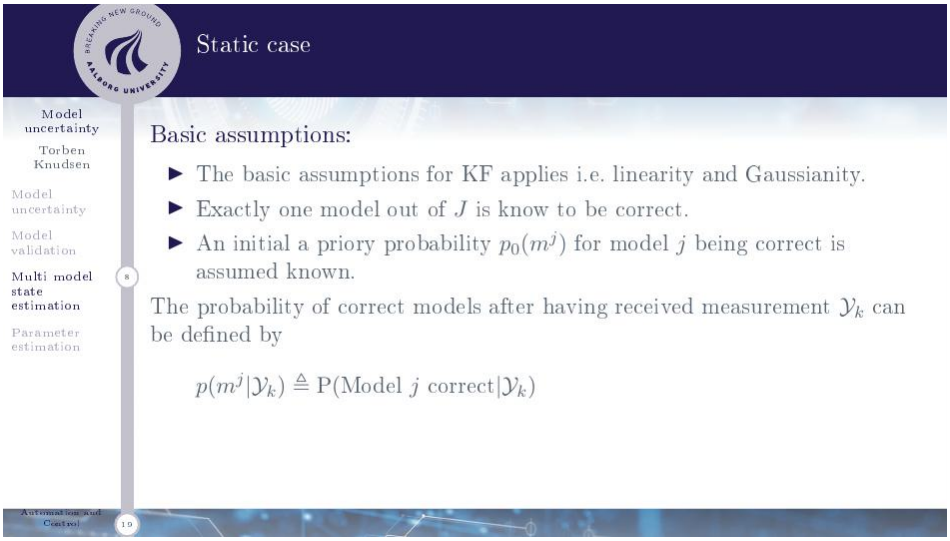

Figure 7: Static case assumptions: one model is correct throughout, KF linearity and Gaussianity assumptions hold, and an initial prior probability $p_0(m^j)$ is assigned to each candidate model.

The **static case** assumes that exactly one model is correct and stays correct for the entire run — no switching. This is the simplest setting; the adaptive (switching) case is in Section 3.7.

Assumptions:

- The standard Kalman filter assumptions hold: linear dynamics, Gaussian noise.
- Exactly one model m^j out of J candidates is the true model.
- An initial **prior probability** $p_0(m^j)$ is known — how likely you believe model j to be correct before seeing any data. If you have no prior preference, set $p_0(m^j) = 1/J$ for all j.

The quantity we track over time is the **posterior probability**:

$$p(m^j \mid \mathcal{Y}_k) \triangleq P(\text{Model } j \text{ is correct} \mid \mathcal{Y}_k)$$

What $\mathcal{Y}_k$ means

$\mathcal{Y}_k$ denotes the complete set of all measurements collected up to and including time step k:

$$\mathcal{Y}_k = \{y_1, y_2, \ldots, y_k\}$$

The posterior $p(m^j|\mathcal{Y}_k)$ is therefore the probability that model j is correct, given *everything we have observed so far*. It starts at the prior $p_0(m^j)$ and is updated every time a new measurement arrives. The model with the highest posterior at time k is our current best guess at the true model.

3.3 Recursive Bayes Update of Model Probabilities

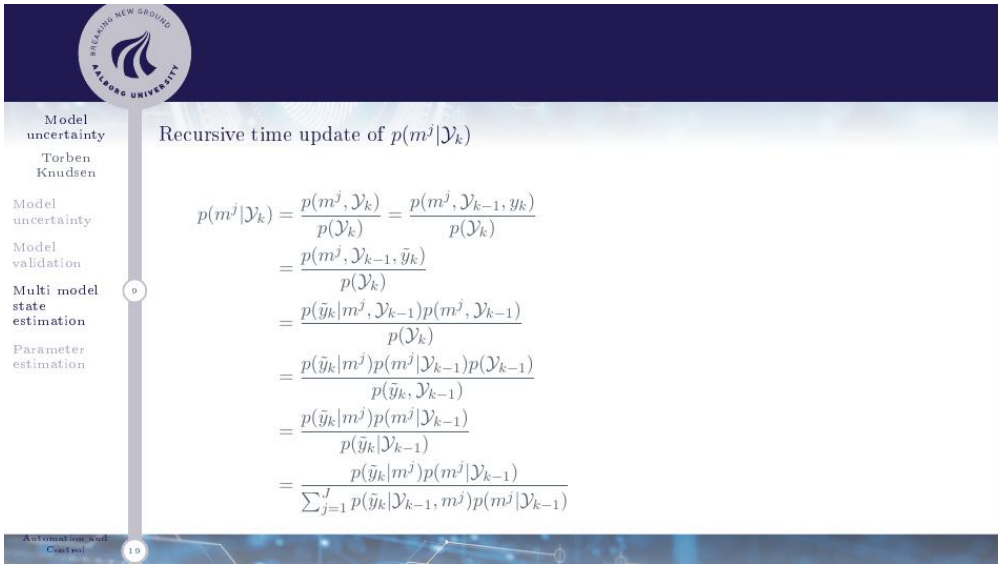

Figure 8: Step-by-step derivation of the recursive update for $p(m^j|\mathcal{Y}_k)$ using Bayes' theorem.

When a new measurement y_k arrives, the posterior probability of each model is updated using Bayes' theorem. Starting from the definition:

$$p(m^j \mid \mathcal{Y}_k) = \frac{p(m^j,\, \mathcal{Y}_{k-1},\, y_k)}{p(\mathcal{Y}_k)} = \frac{p(\tilde{y}_k \mid m^j)\; p(m^j \mid \mathcal{Y}_{k-1})}{\sum_{l=1}^{J} p(\tilde{y}_k \mid m^l)\; p(m^l \mid \mathcal{Y}_{k-1})}$$

where $\tilde{y}_k = y_k - \hat{y}_{k|k-1}$ is the **innovation** (prediction error) of the Kalman filter for the current measurement.

What the formula actually says — plain English

The entire derivation is just one application of Bayes' theorem. The result:

$$\boxed{p(m^j \mid \mathcal{Y}_k) = \frac{p(\tilde{y}_k \mid m^j)\; p(m^j \mid \mathcal{Y}_{k-1})}{\sum_{l=1}^{J} p(\tilde{y}_k \mid m^l)\; p(m^l \mid \mathcal{Y}_{k-1})}}$$

has three pieces:

Numerator — two probabilities multiplied:

- $p(m^j \mid \mathcal{Y}_{k-1})$: how much did we trust model j *before* seeing the new measurement? This is just the previous step's posterior — carried forward.
- $p(\tilde{y}_k \mid m^j)$: how likely is the innovation we just observed, *if* model j is correct? A model that fits the data well gives a high value here; a model that was wrong about what the measurement would look like gives a low value.

Multiplying the two together gives the joint probability: "model j is correct *and* we observe this innovation."

Denominator — normalisation: The denominator is exactly the numerator summed over *all* J models. This is the total probability of observing $\tilde{y}_k$ regardless of which model is true. Dividing by it ensures all J posterior probabilities sum to 1.

In one sentence: the new probability of model j is proportional to how well it predicted the latest measurement, weighted by how much we already trusted it.

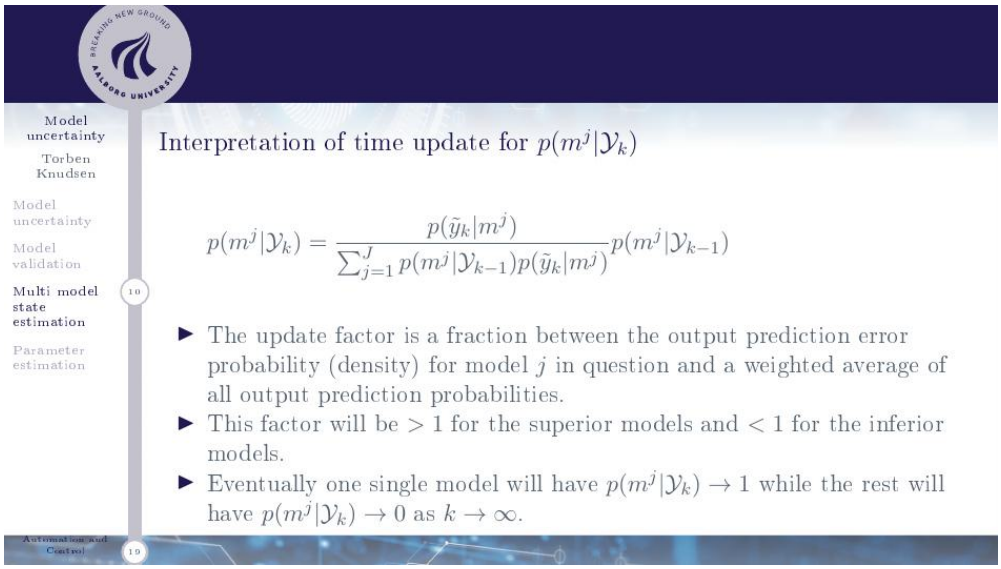

Figure 9: Interpretation of the update: the factor by which each model's probability is multiplied is greater than 1 for models that predicted the current measurement well, and less than 1 for those that predicted it poorly. Over time, one model's probability converges to 1 and the rest to 0.

What the update factor does — intuition

Rewrite the update as:

$$p(m^j \mid \mathcal{Y}_k) = \frac{p(\tilde{y}_k \mid m^j)}{\underbrace{\sum_{l=1}^{J} p(\tilde{y}_k \mid m^l)\, p(m^l \mid \mathcal{Y}_{k-1})}_{\text{update factor}}} \times p(m^j \mid \mathcal{Y}_{k-1})$$

The update factor is the ratio of:

- **Numerator:** how well model j alone predicted the current measurement — the likelihood of the innovation $\tilde{y}_k$ under model j.
- **Denominator:** the weighted average prediction quality across *all* models.

If model j predicted the measurement *better than average*, the factor is > 1 and its probability goes up. If it predicted *worse than average*, the factor is < 1 and its probability goes down.

Why the update factor can exceed 1 — pdf vs probability:

$p(\tilde{y}_k \mid m^j)$ looks like a probability but it is actually a **probability density** (the value of a Gaussian pdf at a specific point). A pdf is not bounded by 1 — a very narrow, precise Gaussian can have a peak value of 10, 100, or more. The denominator is the *weighted average* of these pdf values across all models, which is also an unbounded positive number.

The update factor is therefore a ratio of two pdf values, not two probabilities. If model j's pdf is *above* the weighted average, the ratio exceeds 1 and its probability goes up. If it is *below* the average, the ratio is less than 1 and its probability goes down.

Where the numbers come from: Each model's KF predicts the innovation will follow a Gaussian distribution with mean 0 and covariance P_k^{yj}. A model that is confident and correct has a *tight* Gaussian (small P_k^{yj}) — its curve is tall and narrow. A model that is uncertain or wrong has a *wide* Gaussian (large P_k^{yj}) — its curve is short and flat. $p(\tilde{y}_k \mid m^j)$ is simply the **height of that Gaussian curve at the point where the actual innovation landed**. You read it off the pdf at $\tilde{y}_k$.

Concrete example (3 equal-prior models, innovation = 0):

- Model 1: tight Gaussian (small P^{yj}), tall peak — reading off at $\tilde{y}_k = 0$ gives pdf= 10
- Model 2: medium Gaussian — pdf= 3 at $\tilde{y}_k = 0$
- Model 3: wide or offset Gaussian (large P^{yj} or wrong mean) — pdf= 0.5 at $\tilde{y}_k = 0$

Denominator $= \frac{1}{3}(10 + 3 + 0.5) = 4.5$.
Update factors: $10/4.5 = 2.2,\ 3/4.5 = 0.67,\ 0.5/4.5 = 0.11$.
New probabilities: 0.74, 0.22, 0.04 — still between 0 and 1, still summing to 1, but model 1's factor was > 1.

The final *probabilities* are always between 0 and 1. The intermediate *factors* are not — they are just a convenient way to see which models gain and which lose.

Caution — tight is only good when correct: A tall, narrow Gaussian gives a high pdf value *only near its peak*. If the observed innovation lands far from where model 1 expected (e.g. in its tail), model 1 gives a near-zero pdf value — even lower than the wide models — and gets penalised more severely. High confidence is a double-edged sword: correct predictions are rewarded strongly, but wrong predictions are punished strongly.

Long-run behaviour: the true model consistently predicts measurements well; the others consistently predict poorly. Over time: $p(m^{j^*} \mid \mathcal{Y}_k) \to 1$ for the true model j^*, and $p(m^j \mid \mathcal{Y}_k) \to 0$ for all others as $k \to \infty$.

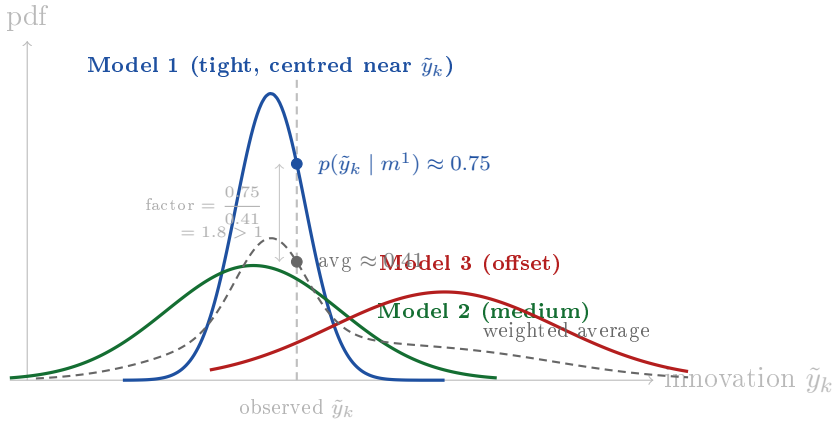

Figure 10: Each model predicts the innovation will follow its own Gaussian curve. $p(\tilde{y}_k \mid m^j)$ is simply the **height of that curve at the point where the actual innovation landed** (vertical dashed line). The dashed black curve is the weighted average of all three — the denominator of the Bayes update. Model 1's tight Gaussian reads off a higher value than the average at this observation point, so its update factor exceeds 1 and its probability increases. *Note:* if the observation had landed far to the right (e.g. at $\tilde{y}_k = 3$), Model 1's narrow curve would give nearly zero there while Model 3's wide curve remains non-negligible — tight is only an advantage when the model is correct.

3.4 Fused State Estimate and Covariance

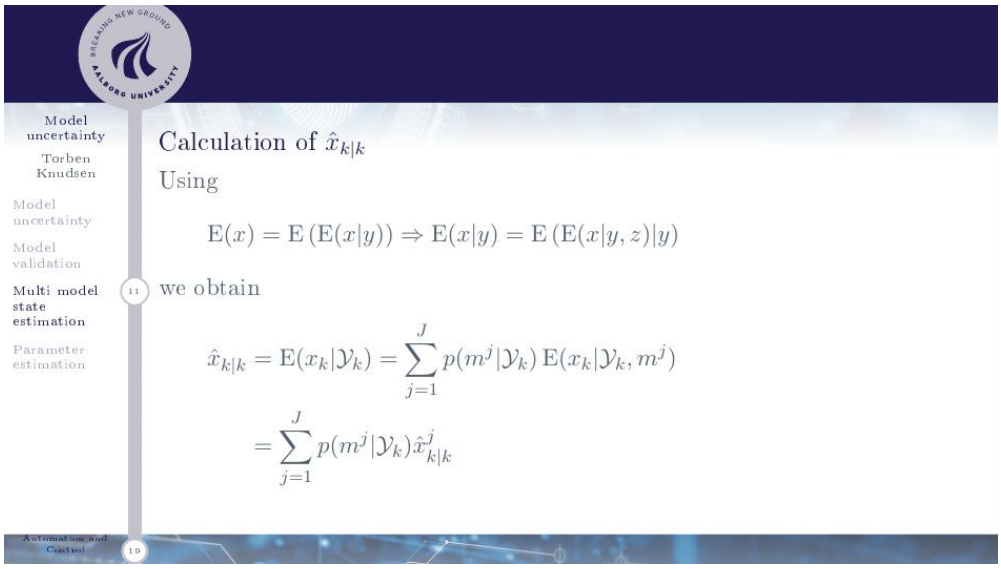

Figure 11: Calculation of the fused state estimate $\hat{x}_{k|k}$ as a probability-weighted average of all model-specific KF estimates.

Each of the J Kalman filters produces its own state estimate $\hat{x}^j_{k|k}$. The overall fused estimate is the **probability-weighted average** over all models, using the *law of total expectation*:

$$\hat{x}_{k|k} = \mathrm{E}(x_k \mid \mathcal{Y}_k) = \sum_{j=1}^{J} p(m^j \mid \mathcal{Y}_k)\ \hat{x}^j_{k|k}$$

If one model has probability near 1, the fused estimate is almost entirely that model's estimate. If two models are equally probable, the fused estimate sits between them.

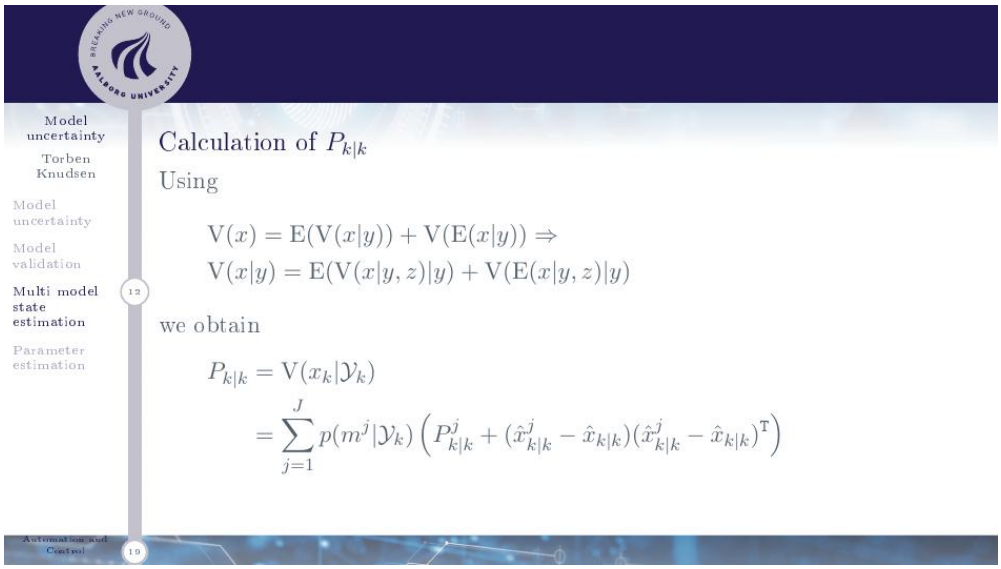

Figure 12: Calculation of the fused covariance $P_{k|k}$. The total covariance has two contributions: the weighted average of each model's own KF covariance, plus the spread of model-specific estimates around the fused estimate.

The fused covariance uses the **law of total variance**:

$$P_{k|k} = \mathrm{V}(x_k \mid \mathcal{Y}_k) = \sum_{j=1}^{J} p(m^j \mid \mathcal{Y}_k)\Big[P^j_{k|k} + \big(\hat{x}^j_{k|k} - \hat{x}_{k|k}\big)\big(\hat{x}^j_{k|k} - \hat{x}_{k|k}\big)^\top\Big]$$

Symbol	Name	Meaning
$\hat{x}^j_{k\|k}$	Model-j state estimate	The KF estimate of the state at time k given all data up to k, using model j.
$P^j_{k\|k}$	Model-j state error covariance	The covariance of the error between the KF state estimate and the true state, assuming model j is correct: $E[(x_k - \hat{x}^j_{k\|k})(x_k - \hat{x}^j_{k\|k})^\top]$. This is exactly the standard KF covariance matrix — the same $P_{k\|k}$ computed in every Kalman filter predict-update step — just one copy per model. Its diagonal entries are the variances of each state component (how far off the x, y, velocity estimates etc. could be); off-diagonals are correlations between state errors.
$\hat{x}_{k\|k}$	Fused estimate	The probability-weighted average across all models.
$P_{k\|k}$	Fused covariance	Total uncertainty including both within-model uncertainty and between-model disagreement.

Why the fused covariance has an extra term

The total uncertainty in the state has two independent sources:

1. Within-model uncertainty $P^j_{k|k}$: suppose you already knew for certain that model j was the correct one. Even then, the KF estimate $\hat{x}^j_{k|k}$ would still have error — because measurements are noisy, the filter may not have fully converged yet, or the process noise keeps the state uncertain. $P^j_{k|k}$ is the KF's own estimate of this irreducible uncertainty. The models may all agree closely on the state (small second term), but if each individual KF is itself uncertain about where the state is (large $P^j_{k|k}$), the total uncertainty is still large. The weighted average $\sum_j p(m^j|\mathcal{Y}_k)\, P^j_{k|k}$ captures this baseline estimation uncertainty.

2. Between-model disagreement $(\hat{x}^j_{k|k} - \hat{x}_{k|k})(\hat{x}^j_{k|k} - \hat{x}_{k|k})^\top$: different models give different state estimates. The fused covariance must account for the fact that we are not certain *which* estimate to trust. Even if each individual KF is very confident (small P^j), if the models disagree on where the state is, the overall uncertainty is large.

Concretely: imagine two models — one estimates position as 5 m, the other as 10 m, each with only 0.1 m uncertainty. The fused estimate might be 7.5 m, but the true uncertainty is not 0.1 m — it is driven by the 5 m gap between the models. The second term captures exactly that spread.

3.5 Computing the Likelihood $p(\tilde{y}_k \mid m^j)$

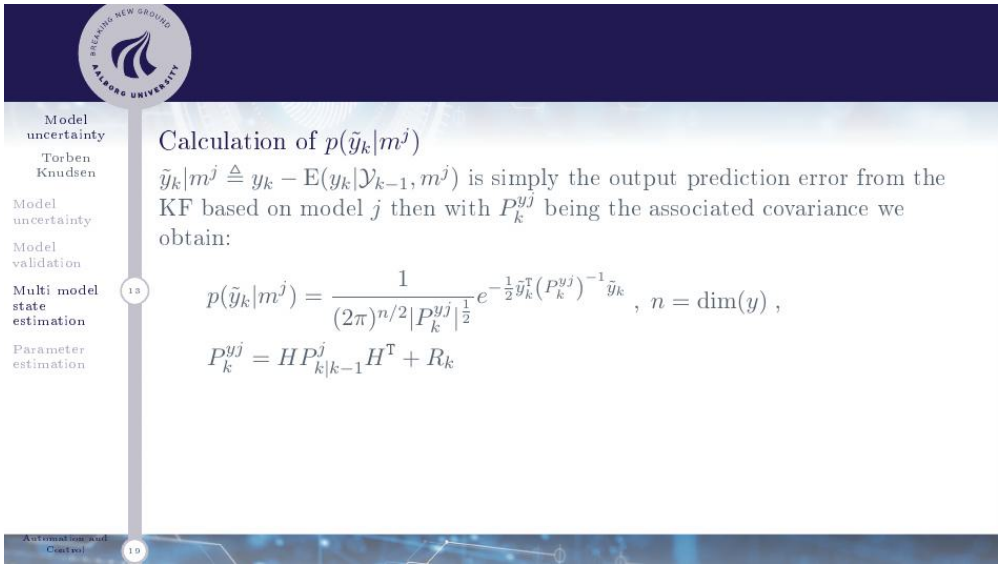

Figure 13: The likelihood $p(\tilde{y}_k \mid m^j)$ is the Gaussian pdf of the KF innovation for model j, evaluated at the observed innovation value.

The Bayes update in Section 3.3 requires $p(\tilde{y}_k \mid m^j)$ — the probability of observing the current innovation under model j. This comes directly from the Kalman filter for model j.

The innovation for model j is:

$$\tilde{y}_k \mid m^j \triangleq y_k - \mathrm{E}(y_k \mid \mathcal{Y}_{k-1}, m^j)$$

which is simply the output prediction error of the KF running with model j — exactly the same residual used in model validation (Section 2).

Under the Gaussian and linearity assumptions the innovation is normally distributed:

$$p(\tilde{y}_k \mid m^j) = \frac{1}{(2\pi)^{n/2}\,|P_k^{yj}|^{1/2}}\ \exp\!\big(-\tfrac{1}{2}\,\tilde{y}_k^\top (P_k^{yj})^{-1}\tilde{y}_k\big)$$

where $n = \dim(y)$ and P_k^{yj} is the innovation covariance for model j:

$$P_k^{yj} = H P_{k|k-1}^j H^\top + R_k$$

Symbol	Name	Meaning
$\tilde{y}_k \mid m^j$	Innovation for model j	The difference between the actual measurement y_k and what model j's KF predicted. A large innovation means model j was surprised by the measurement — it fits the data poorly.
P_k^{yj}	Innovation covariance	The covariance matrix of the innovation under model j — it defines the exact shape of the Gaussian that $\tilde{y}_k$ is expected to follow. The diagonal entries are the variances of each output variable (how much each is expected to fluctuate); the off-diagonal entries are the correlations between output variables. *Large values on the diagonal* of P_k^{yj} mean large variances — model j has wide spread and is not very certain what it will observe, so even a large innovation is not that surprising and does not penalise the model much. *Small values on the diagonal* mean small variances — model j is confident in its prediction, so a large innovation is very unlikely under this model and penalises it heavily. It is computed as $P_k^{yj} = H P_{k\|k-1}^j H^\top + R_k$: the state estimation uncertainty mapped to output space, plus the measurement noise.
H	Observation matrix	Maps the state to the measurement space.
$P_{k\|k-1}^j$	Predicted state covariance	Model j's uncertainty in the predicted state before the measurement update.
R_k	Measurement noise covariance	Noise on the sensor itself.

What this probability actually measures

$p(\tilde{y}_k \mid m^j)$ is high when the observed innovation $\tilde{y}_k$ is small relative to P_k^{yj} — model j predicted the measurement accurately and was not surprised.

$p(\tilde{y}_k \mid m^j)$ is low when $\tilde{y}_k$ is large relative to P_k^{yj} — the actual measurement was far from what model j expected.

This is precisely what drives the probability update: models that are consistently "surprised" by the data have their probability driven toward zero, while models that consistently predict well are driven toward one. Every KF already computes both $\tilde{y}_k^j$ and P_k^{yj} as part of its normal operation — the multi-model extension requires no extra computation beyond evaluating the Gaussian pdf at the existing innovation.

3.6 Block Diagram — Putting It All Together

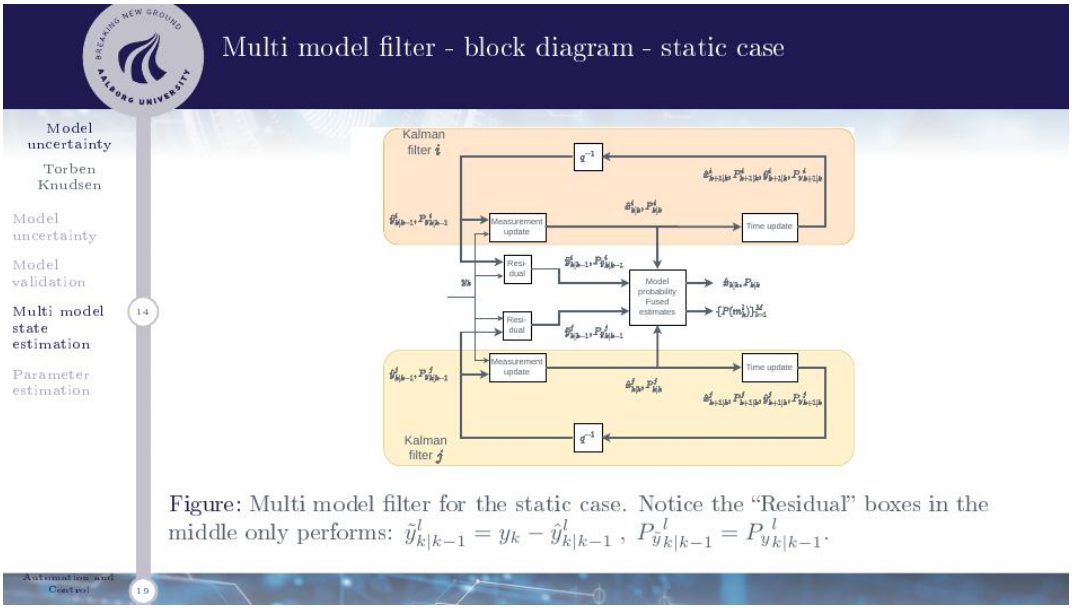

Figure 14: Block diagram of the multi-model filter (static case). J parallel Kalman filters each process the same measurement y_k. Their innovations feed a model probability block that updates $\{p(m^j|\mathcal{Y}_k)\}$. The fused state estimate and covariance are computed from the probability-weighted combination of all KF outputs.

The complete filter runs as follows at each time step k:

1. **Run all J Kalman filters independently.** Every filter sees the same new measurement y_k but uses its own model matrices Φ^j, H^j, Q^j. Each runs a standard

predict-update cycle to produce the four outputs needed by the later steps:

$$\begin{aligned}
\hat{x}^j_{k|k-1} &= \Phi^j \hat{x}^j_{k-1|k-1} && \text{(predicted state)}\\
P^j_{k|k-1} &= \Phi^j P^j_{k-1|k-1}(\Phi^j)^\top + Q^j && \text{(predicted covariance)}\\
\tilde{y}^j_k &= y_k - H^j \hat{x}^j_{k|k-1} && \text{(innovation — how surprised model } j \text{ is)}\\
P^{yj}_k &= H^j P^j_{k|k-1}(H^j)^\top + R_k && \text{(innovation covariance)}\\
K^j_k &= P^j_{k|k-1}(H^j)^\top (P^{yj}_k)^{-1} && \text{(Kalman gain)}\\
\hat{x}^j_{k|k} &= \hat{x}^j_{k|k-1} + K^j_k \tilde{y}^j_k && \text{(updated state estimate)}\\
P^j_{k|k} &= (I - K^j_k H^j)\, P^j_{k|k-1} && \text{(updated state covariance)}
\end{aligned}$$

The key outputs passed to the next steps are: $\tilde{y}^j_k$ (innovation), P^{yj}_k (innovation covariance), $\hat{x}^j_{k|k}$ (updated state estimate), and $P^j_{k|k}$ (updated state error covariance).

2. **Use the innovation $\tilde{y}^j_k$ and innovation covariance P^{yj}_k from step 1 to compute each model's likelihood.** This is the height of model j's Gaussian pdf evaluated at the observed innovation — how well model j predicted what just happened:

$$p(\tilde{y}_k \mid m^j) = \frac{1}{(2\pi)^{n/2}|P^{yj}_k|^{1/2}} \exp\!\left(-\tfrac{1}{2}\,\tilde{y}_k^\top (P^{yj}_k)^{-1}\tilde{y}_k\right), \quad n = \dim(y_k)$$

These J likelihood values $p(\tilde{y}_k \mid m^j)$ feed directly into step 3.

3. **Use the likelihoods $p(\tilde{y}_k \mid m^j)$ from step 2 and the previous model probabilities $p(m^j \mid \mathcal{Y}_{k-1})$ to update the model probabilities via Bayes:**

$$p(m^j \mid \mathcal{Y}_k) = \frac{p(\tilde{y}_k \mid m^j)\; p(m^j \mid \mathcal{Y}_{k-1})}{\displaystyle\sum_{l=1}^{J} p(\tilde{y}_k \mid m^l)\; p(m^l \mid \mathcal{Y}_{k-1})}$$

Models that predicted well gain probability; those that predicted poorly lose it. The updated model probabilities $p(m^j \mid \mathcal{Y}_k)$ feed directly into step 4.

4. **Use the updated model probabilities $p(m^j \mid \mathcal{Y}_k)$ from step 3 together with the updated state estimates $\hat{x}^j_{k|k}$ and state error covariances $P^j_{k|k}$ from step 1 to fuse into the final output:**

$$\hat{x}_{k|k} = \sum_{j=1}^{J} p(m^j \mid \mathcal{Y}_k)\; \hat{x}^j_{k|k} \quad \text{(fused state estimate)}$$

$$P_{k|k} = \sum_{j=1}^{J} p(m^j \mid \mathcal{Y}_k)\Big[P^j_{k|k} + \big(\hat{x}^j_{k|k} - \hat{x}_{k|k}\big)\big(\hat{x}^j_{k|k} - \hat{x}_{k|k}\big)^\top\Big] \quad \text{(fused state error covariance)}$$

The outputs — fused state estimate $\hat{x}_{k|k}$, fused covariance $P_{k|k}$, and updated model probabilities $p(m^j \mid \mathcal{Y}_k)$ — become the inputs for the next time step $k{+}1$.

The "Residual" boxes in the block diagram compute $\tilde{y}^l_{k|k-1} = y_k - \hat{y}^l_{k|k-1}$ and pass the result to the model probability block alongside $P^l_{y,k|k-1}$. Note that the residual block does *not* need the full KF update — it only needs the prediction step output to form the innovation.

3.7 Adaptive Case — Models Changing with Time

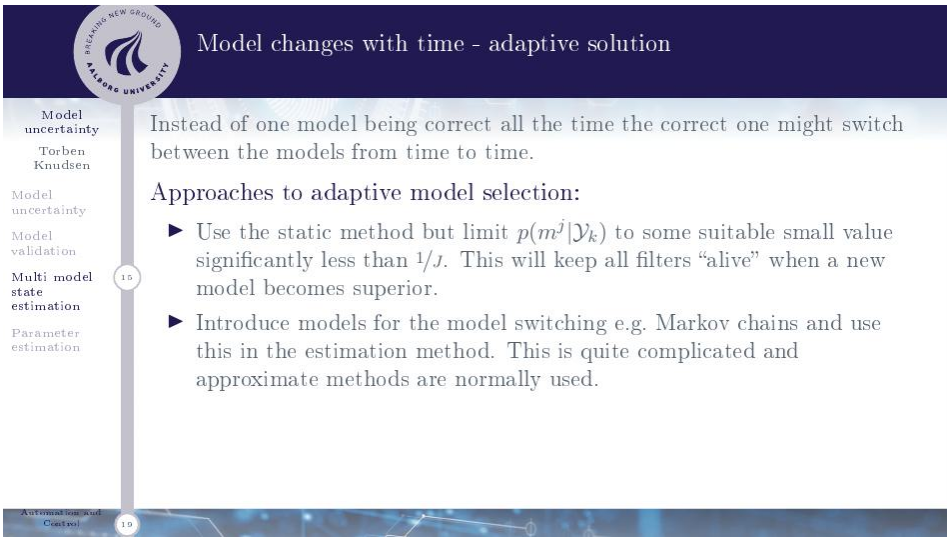

Figure 15: In the adaptive case the correct model may switch between time steps. Two approaches: floor the static-case probabilities to keep all filters alive, or model switching explicitly using Markov chains.

The static case assumes one model is correct forever. In hybrid systems — where the true model can switch (e.g. a vehicle transitioning from dry to icy road) — the static filter eventually kills off all but one model and cannot recover when the switch occurs.

Approach 1 — Probability flooring (simple): Run the static update, but after each step *clip* the model probabilities to a minimum value significantly less than $1/J$:

$$p(m^j \mid \mathcal{Y}_k) \;\geq\; p_{\min} \quad (\text{e.g. } p_{\min} = 0.01 \text{ for } J = 3)$$

What flooring does — and what it does not do

The goal is *not* to keep all probabilities low. One model is still allowed to dominate and be close to 1. The floor is purely a safety net for the *other* models — it prevents any single model from reaching exactly zero and dying.

Without the floor: once a model's probability drops very close to zero, its KF gets negligible weight in the fused estimate and its innovations stop influencing the probability update. It is effectively dead. If the correct model later switches to that dead model, it cannot recover quickly because it has been starved of weight for many steps and needs many new measurements to climb back up.

With the floor: all KFs always receive at least a small weight. They keep running, their innovations keep being computed, and if their model becomes correct again they can rapidly climb back to dominance. The trade-off: the dominant model can never quite reach probability 1, but in a switching system this is the right choice.

Approach 2 — Markov chain model (more accurate): Introduce a **transition matrix** Π where entry Π_{jl} is the probability that the correct model switches *from* model l *to* model j in one time step.

What the Markov chain changes — and what it does not

The individual Kalman filters are *not* modified. All J KFs continue running with their fixed model parameters Φ^j, H^j, Q^j throughout. The KF itself does not detect any switch.

What changes is only the **probability prediction step**. In the static case, before a new measurement arrives, the model probabilities are simply carried forward unchanged. With the Markov chain, the probabilities are first *predicted forward* using the transition matrix:

$$p_{\text{pred}}(m^j \mid \mathcal{Y}_{k-1}) = \sum_{l=1}^{J} \Pi_{jl}\, p(m^l \mid \mathcal{Y}_{k-1})$$

This allows some probability to "bleed" from every model to every other model before the measurement update, so no model ever collapses to zero on its own. The Bayes update (step 3 in Section 3.6) then runs as usual using these predicted probabilities.

In practice the transition matrix Π needs to be designed by the user to reflect how fast and how often the true model is expected to switch — which is often unknown, making approximate methods necessary.

Section 3 — Key Takeaways

- Run J parallel Kalman filters, one per candidate model. Track the **posterior probability** $p(m^j|\mathcal{Y}_k)$ of each model being correct.
- Probabilities are updated by Bayes' rule: $p(m^j|\mathcal{Y}_k) \propto p(\tilde{y}_k|m^j) \cdot p(m^j|\mathcal{Y}_{k-1})$. Models that predict measurements well gain probability; those that predict poorly lose it. Over time the true model converges to probability 1.
- The likelihood $p(\tilde{y}_k|m^j)$ is the Gaussian pdf of the KF innovation — already computed as part of normal KF operation.
- Fused estimate: $\hat{x}_{k|k} = \sum_j p(m^j|\mathcal{Y}_k)\, \hat{x}^j_{k|k}$. Fused covariance adds a **between-model spread term** on top of each model's own KF covariance.
- **Static case:** one model stays correct forever. **Adaptive case:** floor probabilities to keep all filters alive when models can switch.

Section 4 Parameter Estimation

Section 3 handled model uncertainty by choosing between a small discrete set of candidate models. Section 4 handles a different situation: you have **one model structure** that you believe is correct, but some of its numerical parameters are unknown, poorly known, or slowly changing over time.

Rather than picking between fixed alternatives, the goal here is to **estimate the parameter values directly from data**.

4.1 Overview — When Parameters Need Estimating

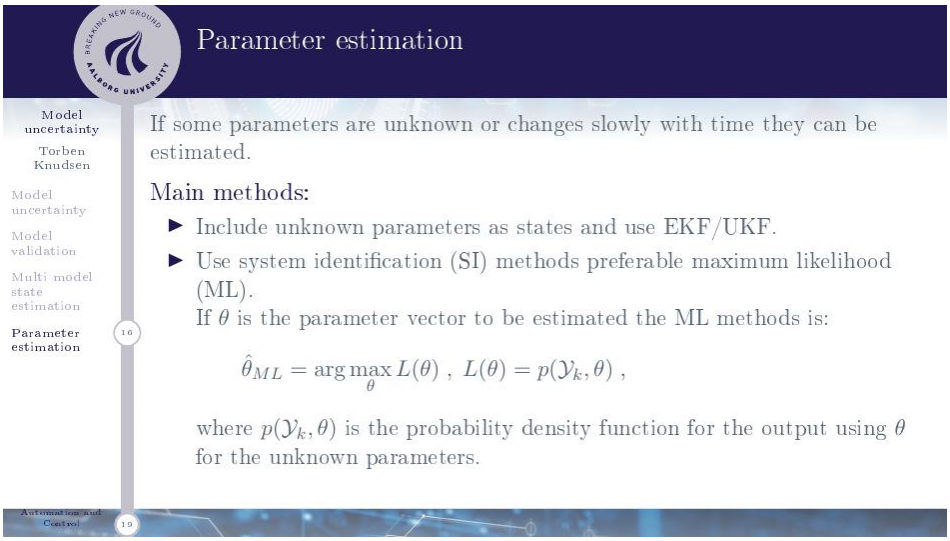

Figure 16: Two main approaches to parameter estimation: augmenting the state vector and using EKF/UKF, or using system identification methods based on maximum likelihood.

If the model structure is correct but a parameter vector θ is unknown or drifting slowly, there are two main strategies:

- **Include θ as extra states and use EKF/UKF.** Augment the state vector with the unknown parameters and run an extended or unscented Kalman filter that estimates both the physical states and the parameters simultaneously, online in real time. Covered in Section 4.2.
- **System identification (SI) — maximum likelihood.** Treat the parameters as fixed unknowns and find the values that make the observed data most probable. This is done offline (using a collected dataset) and gives a single best-fit parameter set rather than a time-varying estimate. Covered in Sections 4.3–4.4.

4.2 Method 1 — Parameters as Augmented States

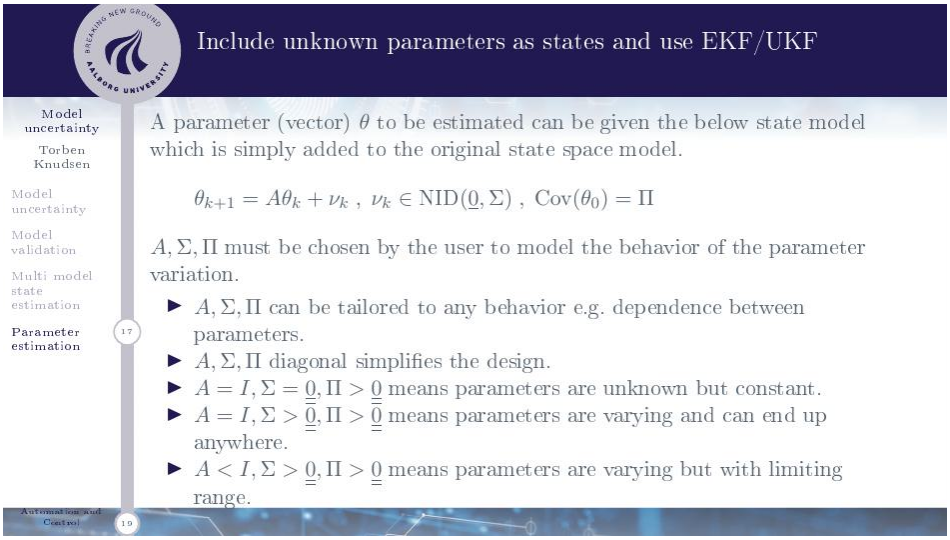

Figure 17: The parameter vector θ is given its own state-space model $\theta_{k+1} = A\theta_k + \nu_k$ and appended to the physical state, allowing joint estimation of states and parameters.

The idea is to append the unknown parameter vector θ to the physical state vector and then run a filter on the combined system. The parameter dynamics are described by an additional state model:

$$\theta_{k+1} = A\,\theta_k + \nu_k, \qquad \nu_k \in \mathrm{NID}(\mathbf{0},\, \Sigma), \qquad \mathrm{Cov}(\theta_0) = \Pi$$

The matrices A, Σ, and Π are chosen by the user to describe how the parameter is expected to behave:

Choice	Behaviour	Physical meaning
$A = I,\ \Sigma = 0,\ \Pi > 0$	Unknown but constant	The parameter is fixed and unknown. The filter estimates it once and does not let it drift. The initial covariance Π reflects how uncertain you are about its value at the start.
$A = I,\ \Sigma > 0,\ \Pi > 0$	Slowly varying, unbounded	The parameter is allowed to drift in a random walk. It can end up anywhere over time. Use when you know the parameter changes but have no idea about its long-run value (e.g. a slowly corroding component).
$A < I,\ \Sigma > 0,\ \Pi > 0$	Slowly varying, bounded range	$A < I$ introduces a restoring force toward zero, so the parameter wanders but stays within a limited range. Use when you believe the parameter varies but cannot drift arbitrarily (e.g. a temperature-dependent coefficient that stays within physical limits).

Why EKF or UKF — not the regular KF?

Once θ is appended to the state, the combined state vector is $[x^\top,\ \theta^\top]^\top$. The original system equations typically involve θ *multiplied by* state variables — for example, $\dot{x} = A(\theta)\,x$. This product makes the combined dynamics **nonlinear** in the augmented state, even if the original system was linear in x alone.

A standard Kalman filter cannot handle this. The **EKF** linearises the combined dynamics at the current estimate using a Jacobian; the **UKF** propagates a set of sigma points through the exact nonlinear equations. Both can track slowly varying parameters — how well depends on how nonlinear the coupling is and how fast the parameter changes relative to the sampling rate.

4.3 Method 2 — Maximum Likelihood Estimation

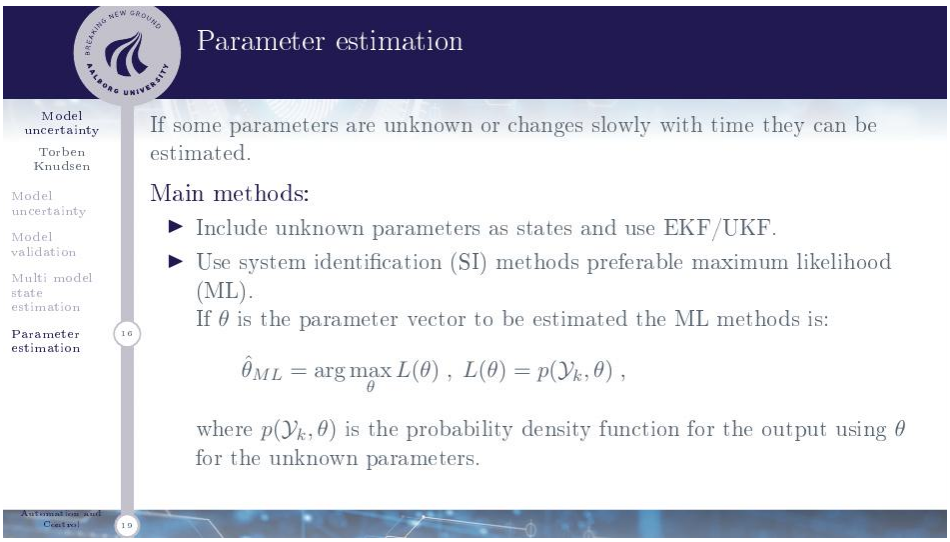

Figure 18: Maximum likelihood: find the parameter vector θ that maximises the probability of observing the collected dataset.

If the parameters are *fixed* (not time-varying), a cleaner offline approach is **maximum likelihood (ML)** estimation. Given a dataset $\mathcal{Y}_N = \{y_1, \ldots, y_N\}$, the ML estimate is:

$$\hat{\theta}_{ML} = \arg\max_{\theta}\ L(\theta), \qquad L(\theta) = p(\mathcal{Y}_N,\, \theta)$$

Symbol	Name	Meaning
θ	Parameter vector	The unknown model parameters to be estimated (e.g. damping coefficient, time constant, sensor gain).
$L(\theta)$	Likelihood function	The probability of observing the entire dataset $\mathcal{Y}_N$ given a specific choice of θ. High L means θ fits the data well; low L means it does not.
$\hat{\theta}_{ML}$	ML estimate	The value of θ that makes the observed data most probable — the best-fitting parameter set.

What maximising the likelihood means in plain terms

Method 2 does still use a Kalman filter, but in a completely different role to Method 1.

In **Method 1** (augmented state), the KF *estimates* θ directly — θ is part of the state vector and the filter updates it every step.

In **Method 2** (ML), the KF is only used as an *evaluation tool* inside an outer optimisation loop. You fix θ at some trial value, run a standard KF (no augmentation), and compute the innovations $\tilde{y}_{k|k-1}$. Those innovations tell you how surprised the model was by the data under that particular θ. If θ is correct, the innovations should be white noise with covariance P_k^y. The likelihood $L(\theta)$ measures exactly how well they match that expectation.

The outer optimiser (e.g. gradient ascent, Nelder-Mead) then tries a new θ, runs the KF again, computes L again, and repeats until it finds the θ that makes the innovations look most like white noise. That is the ML estimate. The KF never estimates θ — the optimiser does, by searching over the space of possible values.

4.4 System Identification Methods — Taxonomy

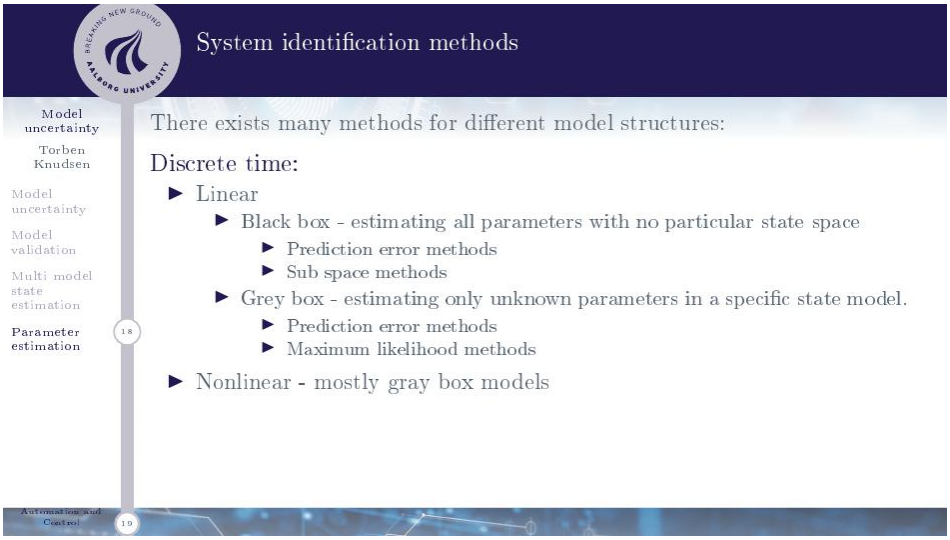

Figure 19: Overview of system identification method families. The choice of method depends on model structure (black box vs grey box), linearity, and whether the system is modelled in discrete or continuous time.

System identification (SI) covers a broad family of methods for fitting model parameters to data. The choice of method depends on what you know about the model structure:

Discrete-time, linear:

- *Black box* — you have no physical model; estimate all parameters of a general ARX/ARMAX/state-space structure with no particular physical meaning. Methods: prediction error methods (PEM), subspace methods.
- *Grey box* — you have a physical model with known structure but unknown parameter values. Methods: prediction error methods, maximum likelihood.

Discrete-time, nonlinear: Mostly grey box models since a black-box nonlinear parameterisation would require an enormous number of parameters.

Continuous-time: Mostly grey box for linear and mildly nonlinear systems. The likelihood function is evaluated using the **continuous-discrete Kalman filter (CD-KF)**, which handles systems specified with continuous dynamics $\dot{x} = f(x, \theta) + w$ but discrete measurements. The CD-KF computes the innovations and their covariances in the same way as the standard KF — the only difference is how the state is propagated between measurement steps (using continuous-time integration rather than a discrete matrix multiply). The likelihood is then assembled from those innovations exactly as in Section 4.5 below.

4.5 Computing the Likelihood via KF Innovations

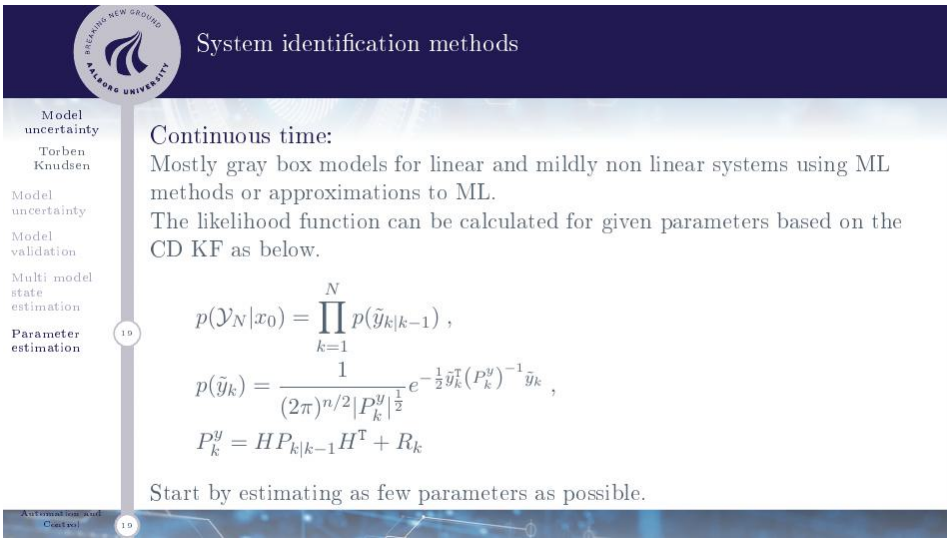

Figure 20: The likelihood factorises into a product of Gaussian pdfs of the KF innovations, one per time step, with innovation covariance $P_k^y = HP_{k|k-1}H^\top + R_k$. This formula applies to both discrete-time and continuous-time (CD-KF) systems.

For both discrete-time and continuous-time systems, the likelihood factorises into a product over time steps:

$$L(\theta) = p(\mathcal{Y}_N \mid \theta) = \prod_{k=1}^{N} p\big(\tilde{y}_{k|k-1} \mid \theta\big)$$

Each factor is the Gaussian pdf of the innovation at step k:

$$p(\tilde{y}_{k|k-1}) = \frac{1}{(2\pi)^{n/2}\,|P_k^y|^{1/2}} \exp\!\big(-\tfrac{1}{2}\,\tilde{y}_{k|k-1}^\top\,(P_k^y)^{-1}\,\tilde{y}_{k|k-1}\big)\,, \quad n = \dim(y)$$

where $P_k^y = HP_{k|k-1}H^\top + R_k$ is the innovation covariance.

Why the likelihood factorises into a product of innovations

This follows from the fact that given the model and all previous measurements, the next innovation is conditionally independent of the ones before it. Each innovation carries one step's worth of "new information" about the data. Taking the product over

all N steps gives the total probability of having seen the entire sequence.

In practice it is more numerically stable to work with the **log-likelihood**:

$$\ln L(\theta) = -\frac{1}{2}\sum_{k=1}^{N}\Big[n\ln(2\pi) + \ln|P_k^y| + \tilde{y}_{k|k-1}^{\top}\,(P_k^y)^{-1}\,\tilde{y}_{k|k-1}\Big]$$

The product becomes a sum, which avoids underflow when multiplying many small probabilities together. Maximising $\ln L$ is equivalent to maximising L since ln is monotone.

Black box vs grey box — the key distinction

A **black-box** model makes no assumptions about the physical process. It fits a general mathematical structure (polynomial, rational function, neural network) purely to reproduce the input-output behaviour. This requires many parameters and gives no physical insight.

A **grey-box** model starts from the physical equations of motion (derived from Newton's laws, circuit theory, thermodynamics, etc.) and only estimates the unknown numerical coefficients (masses, damping ratios, time constants). Far fewer parameters need to be estimated, the estimates have direct physical meaning, and the model generalises better outside the training data. For engineering applications where a physical model exists, grey-box identification is almost always preferable.

Practical advice: start by estimating as few parameters as possible. Each additional unknown parameter makes the optimisation harder and increases the risk of finding a local maximum of $L(\theta)$ that does not represent the true physical values.

Section 4 — Key Takeaways

- Parameter estimation addresses a different problem than multi-model estimation: one model structure, unknown continuous parameter values rather than a discrete set of candidates.
- **Method 1 — augmented state:** append θ to the state vector with dynamics $\theta_{k+1} = A\theta_k + \nu_k$. Choose A, Σ, Π to reflect whether θ is constant ($\Sigma = 0$), a random walk ($A = I$), or mean-reverting ($A < I$). EKF or UKF required because θ enters the dynamics nonlinearly.
- **Method 2 — maximum likelihood:** $\hat{\theta}_{ML} = \arg\max_\theta\, p(\mathcal{Y}_N, \theta)$. Run the KF for each candidate θ and evaluate $L(\theta) = \prod_k p(\tilde{y}_{k|k-1})$ — the product of Gaussian innovation pdfs. Use log-likelihood in practice to avoid numerical underflow.
- Black-box SI estimates all parameters with no physical structure; grey-box SI fits unknown coefficients in a physically-derived model. Grey box is preferred in engineering: fewer parameters, physical meaning, better generalisation.
- **Practical rule:** estimate as few parameters as possible to avoid local optima and identifiability problems.

Lecture Summary — Three Approaches to Model Uncertainty

Approach 1 — Model Validation (Section 2): "Is the model any good?"

This approach does not fix anything — it *diagnoses*. You run the Kalman filter with the model you have and collect the residuals (innovations) $\tilde{y}_{k|k-1}$ over a long dataset. Then you check whether those residuals have the statistical properties they *should* have if the model were correct: zero mean, uncorrelated over time (white noise), and Gaussian.

If the tests pass, you have evidence the model is at least statistically consistent with the data. If they fail — residuals are correlated, non-Gaussian, or biased — the model is missing something and can be improved. The tests tell you *that* there is a problem; they do not tell you *what* to fix. Tools: run test (sign changes), ACF / Portmanteau test, normal probability plot.

Approach 2 — Multi-Model Estimation (Section 3): "Which of these candidate models is correct?"

This approach is used when you have a small discrete set of candidate models — different suppliers, different parameter values, different complexity levels — and you do not know which is the true one. Rather than picking one upfront, you run *all* of them in parallel, each as a complete Kalman filter.

At every time step, you ask each model: how well did you predict the measurement that just arrived? Models that are consistently surprised (large innovations relative to their expected spread) lose probability; models that predict well gain probability. This is a Bayesian update: the probability of each model being correct is updated using the ratio of how well that model fit the new data versus the weighted average fit across all models.

The final state estimate is a probability-weighted blend of all model-specific estimates. Over time, the true model's probability converges to 1 and the others go to 0 — the data has effectively selected the correct model for you.

Approach 3 — Parameter Estimation (Section 4): "What is the value of this unknown parameter?"

This approach is used when the model structure is correct but some continuous parameter values are unknown. There are two sub-methods:

Online (augmented KF): Append the unknown parameter θ to the state vector and run an EKF or UKF on the combined system. The filter estimates both the physical states and θ simultaneously at every time step. No separate optimisation loop is needed — the KF does everything.

Offline (maximum likelihood): Here the KF and an optimiser work together in a

loop. The *optimiser* (e.g. gradient ascent, Nelder-Mead) proposes a trial value of θ. The *KF* is run with that fixed θ and produces innovations $\tilde{y}_{k|k-1}$ — how surprised the model was at each step. The likelihood $L(\theta) = \prod_k p(\tilde{y}_{k|k-1})$ measures how well that θ fits the entire dataset. The optimiser uses this score to decide which direction to step next — toward θ values that make the KF *less* surprised by the data (smaller, better-calibrated innovations). The KF acts as an indicator of fit quality; the optimiser uses that signal to search for the best θ. The loop repeats until the likelihood stops improving.